

Clase 4_2 :

Programación

Programación Orientada a
Objetos

Temario

- Programación Orientada a Objetos en Python
 - Definición de clases
 - Definición de atributos
 - Definición e Invocación de métodos
 - Constructores
- Encapsulamiento y ocultamiento de información



Programación Orientada a Objetos con Python

POO en Python

- Python está completamente preparado para soportar la programación orientada a objetos.
- En Python cuando se utiliza la programación orientada a objetos se puede:
 - Definir **nuevas clases**.
 - Utilizar **mecanismos** para definir **relaciones de herencia entre clases**.
 - **Instanciar las clases** ya definidas en el sistema o las que haya definido el desarrollador.

Clases en Python

- Una clase de Python empieza con la palabra reservada **class**, seguida de su **nombre** y finalizando con **:** (**dos puntos**)
 - **class Auto:**
 - **class Persona:**
- Por convención los nombres de clases deben comenzar en mayúscula.

Clases en Python

- **Dentro de la clase** se especifican los **métodos y los atributos comunes a todos los objetos** de dicha clase.
- **Todo** lo que va **dentro de una clase** se escribe con **sangría o indentación**.
- Las clases **pueden ser definidas en un módulo independiente** o bien **dentro del módulo de la aplicación** que va a utilizar dicha clase. En este último caso, la primera sentencia que no esté con sangría, no pertenece a la clase.

clase libro en Python

- Definamos la clase **Libro**

```
class Libro:
    def __init__(self, unNombre, unAutor, unPrecio):
        self.__nombre = unNombre
        self.__autor = unAutor
        self.__precio = unPrecio

    def getNombre(self):
        return self.__nombre

    def setNombre(self, nombre_libro):
        self.__nombre = nombre_libro

    def __str__(self):
        return "Nombre: " + self.getNombre()
```

Nombre de la clase

Cuerpo de la
clase Libro

Variables de
instancia
(ATRIBUTOS!!!)

Métodos
COMPORTAMIENTOS

Instancias en Python

- Para crear un objeto hay que instanciarlo a partir de una clase. Para ello se utiliza la notación de funciones, o sea se escribe el nombre de la clase seguido por dos paréntesis, uno de apertura y otro de cierre

```
libro1 = Libro("Los 7 Locos", "Roberto Arlt",1200)
libro2 = Libro("Los Lanzallamas", "Roberto Arlt",1300)
libro3 = Libro("El Jugete Rabioso", "Roberto Arlt",1500)
```

Creación de **tres instancias** de la clase Libro. Se invoca en forma automática al constructor. **Se crean 3 objetos "Libro"**

Constructores

- Los constructores son métodos especiales mediante los cuales los programadores pueden inicializar una instancia. Pueden recibir o no parámetros. Si tienen parámetros, dichos valores se usan para inicializar las variables de instancia del objeto creado.
- Los constructores se ejecutan automáticamente justo después de crear o instanciar un objeto.
- Los constructores en Python se definen codificando un método especial llamado `__init__` . (lleva doble guión)
- En el `__init__` se inicializan las variables de instancia del objeto, dejándolas disponibles para comenzar a operar con ellas a través de los métodos.

Constructores

- Ejemplo de la implementación de un constructor para inicializar una variable de instancia en los objetos Libro .

```
class Libro:  
    def __init__(self, unNombre, unAutor, unPrecio):  
        self.__nombre = unNombre  
        self.__autor = unAutor  
        self.__precio = unPrecio
```

Constructor de la clase Libro

Inicializa el atributo `__precio` con el valor contenido en el parámetro `unPrecio`.

Self

- La palabra reservada **self** sirve para referirse al objeto actual, que es el que recibe el mensaje. Es utilizada dentro del método para señalarse a sí mismo y sirve para poder acceder a los atributos y métodos del objeto actual.
- En todos los métodos de una clase el primer parámetro es siempre **self**.

```
class Libro:

    def __init__(self, unNombre, unAutor, unPrecio):
        self.__nombre =
        unNombre self.__autor =
        unAutor self.__precio
        = unPrecio
```

Nos permite especificar que nombre, autor y precio son las variables de instancia.

Atributos en Python

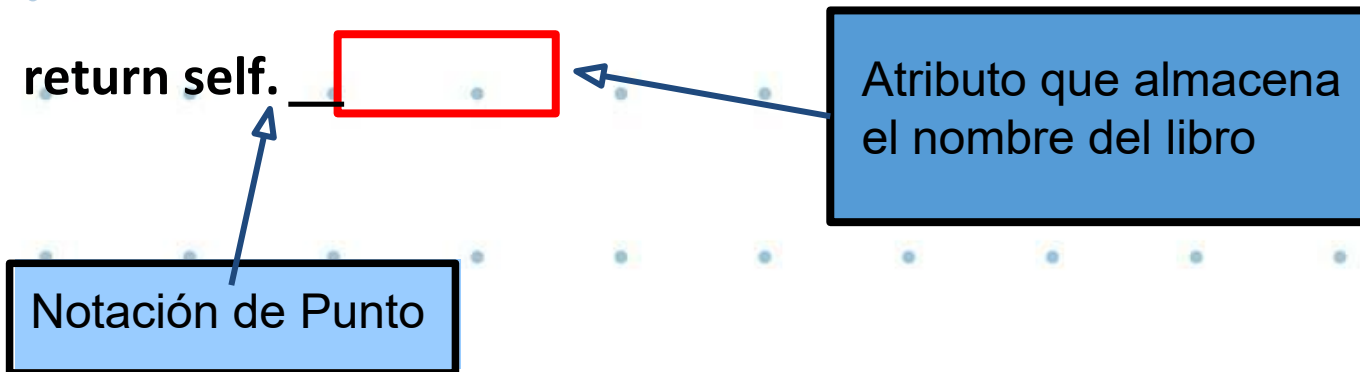
- Los *atributos o variables de instancia* permiten almacenar el estado de un objeto.
- Para asignar o acceder a la información que se almacena en ellos se utiliza la notación de punto.

return self.



Atributo que almacena
el nombre del libro

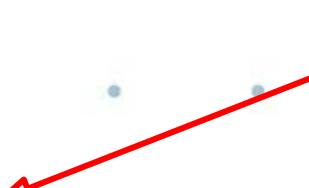
Notación de Punto



Atributos en Python

```
class Libro:
```

Referencia al
objeto actual



```
def __init__(self, unNombre, unAutor, unPrecio):
```

```
    self.__nombre = unNombre
```

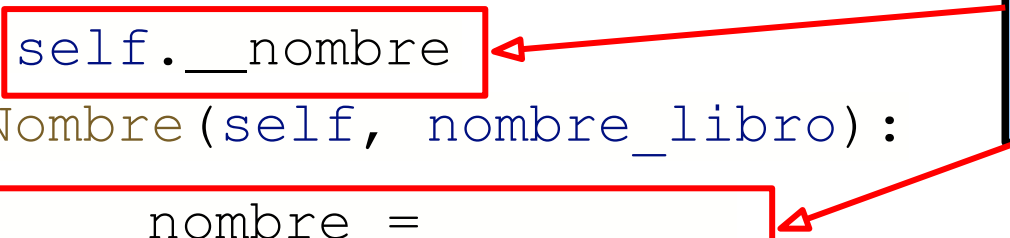
```
    self.__autor =
```

```
    self.__precio =
```

```
def getNombre(self):
```

```
    return self.__nombre
```

Referencia al
atributo o
variable de
instancia



```
def setNombre(self, nombre_libro):
```

```
    self.__nombre =
```

```
    nombre_libro
```

Declaración de Métodos

- Los **métodos** en Python son muy **similares** a las **funciones**.
- Las **similitudes** más significativas son:
 - Para definir un método:
def nombreMetodo(parámetros):
 - El cuerpo debe estar indentado.
 - Pueden o no retornar un resultado.
- Pero **se diferencian** de las funciones en que:
 - Los **métodos** se encuentran **dentro de una definición de clase**.
 - El primer parámetro siempre debe ser **self** aunque no se lo utilice luego en el método.

Declaración de Métodos

```
class Libro:
    def __init__(self, unNombre, unAutor,
unPrecio):
    self.__nombre =
unNombre
    self.__autor = unAutor
    self.__precio =
unPrecio

    def getNombre(self):
        return self.
        __nombre

    def setNombre(self, nombre_libro):
        self.__nombre =
        nombre_libro

    def precioEnDolares(self, valor_dolar):
        return self.__precio /
        valor_dolar

    def __str__
        return "Nombre: " + self.getNombre()
```

Dos nuevos
métodos

Invocación de Métodos

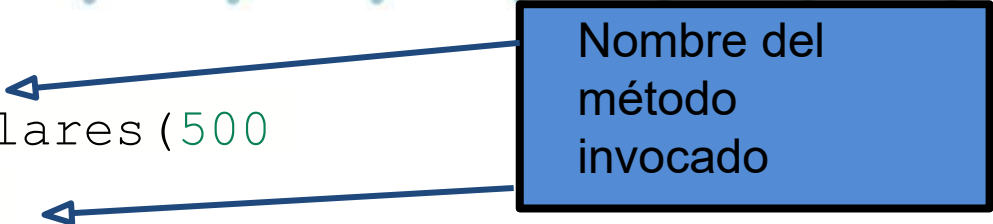
La forma en que **los métodos son invocados** difiere de la forma de invocar a las funciones. **Se debe utilizar la notación de punto.**

- **Al invocar un método el parámetro `self` se omite ya que está implícito en el objeto.**

```
libro1 = Libro("Los 7 Locos", "Roberto  
Arlt", 1200)
```

```
libro1.precioEnDolares(500
```

```
libro1.getNombre(  
)
```



Nombre del
método
invocado

Miembros

- En POO se denomina:
- **Datos miembros, a las variables de instancia**
- **Funciones miembros, a los métodos**
- Hay dos **tipos de miembros**, tanto para datos como para métodos:
 - **De instancia**
 - **De clase**

Por lo cual se pueden definir:

- * **variables de instancia y métodos de instancia**
- * **variables de clase y métodos de clase**

Miembros de instancia

Supongamos tener creada la clase
Auto

```
class Auto:
```

```
    def __init__(self,mar,mod):  #constructor  
        self.__marca=mar  
        self.__modelo=mod
```

Variables de instancia

```
def verMarca(self):  
    return self.__marca
```

#método de instancia

.....

Miembros de instancia

- Los **miembros de instancia**, ya sean variables o métodos, **son utilizados cuando se trabaja con instancias de una clase**

```
a1= Auto("Ford",1987)
```

```
a2=
```

```
Auto("Peugeot",1995)
```

```
a3= Auto("Fiat",2008)
```

```
print ('Marca:', a1.verMar())
```

```
print ('Marca:', a2.verMar())
```

```
print ('Marca:', a3.verMar())
```

a1 , a2 y a3 son instancias de la clase Auto.

verMar() es un **método de instancia**, se aplica **sobre objetos** o instancias de la clase Auto. Retorna el contenido de la variable de instancia

marca

Miembros de instancia

- Cuando se instancia una clase, se genera en memoria una "copia" de la clase para cada instancia creada.

RAM

a1

marca = "Ford"
modelo = 1987

a2

marca = "Peugeot"
modelo = 1995

a3

marca = "Fiat"
modelo = 2008

Técnicamente hay tres variables **marca** y tres variables **modelo**. Son las variables de instancia de cada objeto

Miembros de instancia

- Los **métodos de instancia** son los que acceden a **cada variable** dependiendo de quién sea el objeto receptor del mensaje.

RAM

a1

marca = "Ford"
modelo = 1987

a2

marca = "Peugeot"
modelo = 1995

a3

marca = "Fiat"
modelo = 2008

a1.verMar()

El método verMar usará la variable marca de a1

Miembros de instancia

- Los métodos de instancia son los que acceden a cada variable dependiendo de quien sea el objeto receptor del mensaje.

RAM

a1

marca = "Ford"
modelo = 1987

a2

marca = "Peugeot"
modelo = 1995

a3

marca = "Fiat"
modelo = 2008

a2.verMar()

El método verMar usará la variable marca de a2

Miembros de instancia

- Los métodos de instancia son los que acceden a cada variable dependiendo de quién sea el objeto receptor del mensaje.

RAM

a1

marca = "Ford"
modelo = 1987

a2

marca = "Peugeot"
modelo = 1995

a3

marca = "Fiat"
modelo = 2008

a3.verMar()

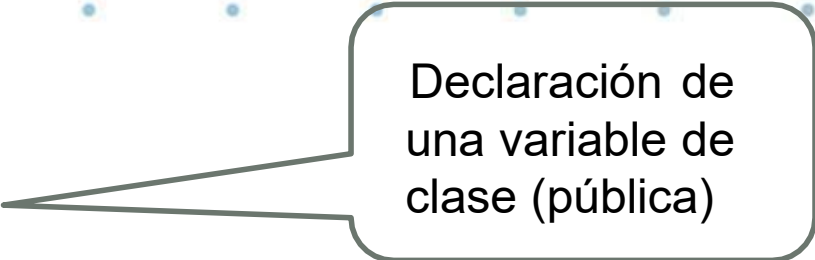
El método verMar usará la variable marca de a3

Miembros de clase

Las variables de clase son elementos que guardan información común y compartida por todas las instancias de una clase.

Se la define luego del encabezado de la clase, como si fuera una variable global. Su nombre comienza con mayúscula. Hay una sola copia de dicha variable.

```
class Auto:  
    Origen='Argentina'  
  
    def __init__(self, mar,  
mod):  
  
        self._marca = mar  
        self._modelo = mod
```



Declaración de una variable de clase (pública)

Miembros de clase

RAM

Auto Origen = 'Argentina'

a1

marca = "Ford"
modelo = 1987

a2

marca = "Peugeot"
modelo = 1995

a3

marca = "Fiat"
modelo = 2008

La variable Origen es una variable de clase y es común a todas las instancias. **Hay una sola copia.** Por eso muestra el valor Argentina para cualquier instancia.

Miembros de clase

Las formas de acceder a una variable de clase son:

-
- `NombreDeClase.nombreVariable`
- `self.nombreVariable`

Esta variable de clase, nos **permite utilizarla sin necesidad de instanciar**, es decir, de crear un objeto perteneciente a esta clase.

Se le puede dar un valor aún cuando no haya **ningún objeto de la clase** creado.

Si está definida como **pública**, se puede acceder a ella solo con colocar el **nombre de la clase y el nombre de la variable**:

```
print ('Fabricado en:', Auto.Origen)
```

Encapsulamiento y ocultamiento

En Python:

- la clase permite encapsular la representación interna y la implementación de los métodos.
- El ocultamiento de un atributo o método viene determinado por su nombre:
 - *si el nombre comienza con dos guiones bajos (y no termina también con dos guiones bajos) se trata de un atributo o método privado,*
 - en caso contrario es público.

Ocultamiento de información en Python

```
class Ascensor:
    → def __init__(self, __pisoActual):
    →     self.__pisoActual = __pisoActual
    →     self.disponible = True

    → def subir(self):
    →     self.__pisoActual = self.__pisoActual + 1

    → def bajar(self):
    →     self.__pisoActual = self.__pisoActual - 1

    → def getPisoActual(self):
    →     return self.__pisoActual

    → def __setPisoActual(self, __pisoActual):
    →     self.__pisoActual = __pisoActual

ascensor1 = Ascensor(0)

ascensor1.disponible = True
ascensor1.getPisoActual()
ascensor1.__pisoActual = __pisoActual
ascensor1.__setPisoActual(2)
```

Miembros Privados

Miembros Públicos



Acceso Válido



Acceso Invalido

Ventajas de la Programación OO

- **Reusabilidad del código** mediante **herencia y encapsulamiento**
- **Fácil comprensión de la lógica del programa**
- **Facilidad en el mantenimiento y expansión**
- **Fácil documentación y diseño del programa**

Ejercicio 1

Defina una **clase llamada "Mascota" con 4 atributos: nombre de la mascota, especie (perro, gato, tortuga, etc.), edad en años y nombre del dueño.**

Defina un constructor que reciba los cuatro campos.

class Mascota:

```
def ____init  
    ____ (self,nom,esp,due,ed):
```

```
    self.__nombre=nom
```

```
    self.  especie=esp
```

```
    self.  duenio=due
```

```
    self.  edad=ed
```

```
def verNom(self):
```

```
    return self.
```

```
    ____nombre
```

```
def verEdad(self):
```

```
    return self.
```

```
    ____edad
```

```
def verEsp(self):
```

```
    return self.
```

```
    ____especie
```

```
def verDuenio(self):
```

} Constructor

Se guarda con el
nombre
clasemascota.p
y

```
def  
    modNom(self, otro):  
        self.__nombre=otro
```

```
def  
    modEd(self, otra):  
        self.__edad=otra
```

```
def  
    modEsp(self, otra):  
        self.__especie=otra
```

```
def  
    modDuenio(self, otro):  
        self.__duenio=otro
```

Ejercicio 2

Dada la definición de la clase **Mascota** del ejercicio anterior ¿cuáles de las siguientes sentencias de instanciación son correctas?

Mascota m

m = new Mascota ("Fufu", "conejo")

m = Mascota ("Fido", "perro")

m = Mascota ("Silver", "caballo", 10, "Llanero")

m = new Mascota ()

m=Mascota('beto','perro','Jorge',5)

Ejercicio 3



Agregue a la clase "Mascota" un método de instancia llamado "hablarConElDueño" que dependiendo de la especie haga su sonido característico (ladrar, maullar, relinchar, etc,)

Ejercicio 3

dentro de la clase agregamos un nuevo método

```
def hablarConElDueño(self):
```

```
    if self.
```

```
        __especie=="perro":
```

```
        hace="Estoy ladrando"
```

```
    elif self.
```

```
        __especie=="gato":
```

```
        hace="Estoy maullando"
```

```
    elif self.
```

```
        __especie=="caballo":
```

```
        hace= "Estoy relinchando"
```

```
    elif self.___especie==
```

```
        "canario": hace="Estoy
```

```
        piando"
```

```
    elif self.
```

```
        especie=="conejo":
```

Ejercicio 4

Haga un programa que diferentes mascotas, las guarde en una lista y luego recorra la colección e haciendo "hablar" a las mascotas

#Aplicación

```
from clasemascota import *          #importa la
```

- `clase listamascotas = []`
- `m= Mascota ("Manuelita", "tortuga", "Elena", 4)`
- `listamascotas.append(m)`
- `m = Mascota ("merlin", "gato", "Pedro", 5)`
- `listamascotas.append(m)`
- `m= Mascota ("Silver", "caballo", "Llanero", 10)`
- `listamascotas.append(m)`

```
for a in listamascotas:  
    print (a.hablarConElDue())  
input()
```

